

PRODUCT REQUIREMENTS DOCUMENT

AI Agent Wallet

Programmable, policy-controlled payments for autonomous AI agents on Solana

PRODUCT THESIS AI proposes a payment; the user retains control; the Solana program enforces the non-negotiable spending rules.

Document field	Value
Version	1.0 — MVP definition
Status	Draft for build planning
Owner	Product & Engineering
Target network	Solana devnet for MVP; mainnet-beta only after audit and operational readiness
Primary currency	USDC (SPL token)
Target release	12-week MVP build

Decision requested

Approve the MVP scope: policy-controlled USDC payments to pre-approved merchants, with AI-assisted payment proposals and explicit user confirmation. Autonomous, unattended payments are deliberately deferred until the security and compliance gates are met.

1. Executive Summary

AI Agent Wallet is a non-custodial payment-control layer for AI agents. A user deposits USDC into a program-derived vault and configures a policy that caps spend, restricts recipients, sets an expiry, and provides an emergency pause/revoke mechanism. An off-chain AI agent can discover and prepare a purchase, but it cannot bypass the on-chain policy or transfer funds outside its approved authority.

1.1 Problem

AI agents are increasingly capable of taking actions, yet they cannot safely be given a conventional wallet private key. A key grants broad authority, creates irreversible-loss risk, and offers limited control over what the agent may buy. Businesses and individual users need a verifiable boundary between an agent’s recommendation and an actual payment.

1.2 Product Vision

Make AI-driven commerce safe enough to use: agents should be able to request and execute narrowly authorized payments, while users can inspect, constrain, and revoke that authority at any moment.

1.3 Goals

- Enforce payment limits and recipient restrictions in a Solana program—not only in backend code.
- Give users a simple workflow to create a policy, fund it with devnet USDC, approve proposals, and inspect receipts.
- Demonstrate a complete AI → policy check → on-chain settlement flow with a mock merchant.
- Provide a clean, testable contract foundation for future merchant, subscription, and delegated-agent integrations.

1.4 Non-goals (MVP)

- Holding user private keys or operating a custodial exchange.
- Real-world fiat checkout, KYC/AML processing, credit cards, or bank rails.
- On-chain LLM inference, model training, or claiming that AI outputs are correct.
- Unattended recurring payments, swaps, cross-chain bridges, lending, or investment advice.
- An open merchant marketplace or arbitrary transfers to any wallet.

2. Users, Jobs, and Value

Persona	Primary job	Pain today	MVP value
Individual builder	Let an AI tool purchase an approved service	Must either manually pay every time or expose a wallet	A spend-limited vault and clear approval trail

Persona	Primary job	Pain today	MVP value
	within a small budget	key	
Team / startup operator	Experiment with agent-run tooling without uncontrolled spend	API/compute buying is hard to delegate safely	Rules are explicit, revocable, and auditable
Demo merchant	Accept a programmatic payment with proof of settlement	Cannot trust an AI agent's off-chain promise	Receives final USDC transfer and receipt
Developer / auditor	Understand and validate payment authority	Backend-only rules are opaque	Anchor program exposes deterministic checks and events

2.1 Core jobs to be done

- When I ask an AI to purchase an approved service, I want the payment confined to rules I choose so I do not expose my entire wallet.
- When a payment is proposed, I want to see the merchant, amount, purpose, and policy impact so I can make an informed approval.
- When a payment is executed, I want an immutable receipt so I can reconcile what my agent did.
- When I no longer trust an agent or merchant, I want to stop future payments immediately.

3. Product Scope and User Experience

3.1 MVP user journey

Step	User experience	System behavior	Success condition
1. Connect	User connects Phantom-compatible wallet	Frontend verifies network and public key	Wallet is connected on devnet
2. Create policy	User enters name, daily limit, per-payment cap, expiry	Client derives policy/vault PDAs; wallet signs initialization	Active policy is created
3. Fund vault	User deposits devnet USDC	Program transfers USDC into policy vault	Vault balance reflects deposit
4. Allow merchant	User adds Demo Merchant from known list	Program records merchant allowlist entry	Merchant is active for policy

Step	User experience	System behavior	Success condition
5. Ask agent	User: “Pay Demo Merchant 2 USDC for image credits”	Agent returns a structured proposal; no funds move	Proposal displays all fields
6. Confirm	User clicks Approve & Pay	Wallet signs execute-payment transaction	Program validates every constraint
7. Receive receipt	Dashboard shows status and transaction link	Program transfers USDC and emits event	Payment appears in history
8. Stop authority	User pauses/revokes policy	Program changes policy status	New payments are blocked immediately

3.2 Key screens

- Onboarding: wallet connect, devnet explanation, and airdrop/test-USDC setup guidance.
- Policy dashboard: vault balance, daily spend, remaining allowance, expiry, policy state, pause/revoke controls.
- Policy editor: daily cap, per-transaction cap, expiry, approved merchants, and confirmation preference.
- Agent chat and proposal card: merchant, amount, currency, purpose, policy checks, remaining budget, and transaction preview.
- Activity page: immutable receipt list with status, transaction signature, merchant, amount, and policy ID.
- Merchant demo checkout: a mock service that creates a signed/order-bound payment request.

UX RULE The default MVP requires a wallet signature for every payment. “Autonomous execution” is a later feature flag, not a default capability.

4. Functional Requirements

ID	Requirement	Priority	Acceptance criterion
FR-01	Connect a Solana wallet and detect devnet	Must	Connected address and network are shown; unsupported networks are blocked.

ID	Requirement	Priority	Acceptance criterion
FR-02	Create one or more policies	Must	User creates a PDA-backed policy with validated limits and expiry.
FR-03	Deposit and withdraw USDC	Must	Deposits reach the policy vault; only policy owner can withdraw unused funds.
FR-04	Manage merchant allowlist	Must	Only the owner can add/remove merchant token accounts; duplicate entries are rejected.
FR-05	Submit a payment proposal	Must	Agent produces structured data and an explanatory UI card; it cannot sign a transfer.
FR-06	Execute an approved payment	Must	Program validates status, owner/signature, recipient, limits, expiry, unique request ID, and balance before SPL transfer.
FR-07	Track daily spending	Must	Spend is accumulated in a deterministic UTC day window; an over-limit payment fails.
FR-08	Pause and revoke policy	Must	Pause blocks execution; revoke permanently disables execution and merchant changes.
FR-09	Display receipts and history	Should	Activity view reflects confirmed transaction data and program events.
FR-10	Show policy simulation before signing	Should	UI displays pass/fail checks and budget remaining; contract remains final authority.
FR-11	Mock merchant fulfillment webhook	Could	Merchant marks a demo order paid after confirmed on-chain transfer.

5. Detailed Rules and Payment State Machine

5.1 Policy rules

Rule	Definition	Enforced where
Policy state	Only ACTIVE policies may pay. PAUSED and REVOKED reject payment instructions.	On-chain
Ownership	Only the owner signer can initialize, fund, manage merchants, pause, revoke, or withdraw.	On-chain
Merchant restriction	Payment destination must match an active policy-specific merchant entry and canonical USDC token account.	On-chain
Per-payment cap	Payment amount must be > 0 and $\leq \text{per_tx_limit}$.	On-chain

Rule	Definition	Enforced where
Daily cap	Current UTC-day spend + amount must be $\leq$ daily_limit; bucket resets when day index changes.	On-chain
Expiry	Current on-chain clock timestamp must be before expires_at.	On-chain
Replay protection	A request_id hash may be used once per policy.	On-chain
Sufficient funds	Vault must contain enough USDC for the transfer.	SPL Token program / on-chain
Proposal integrity	Merchant, amount, currency and purpose shown to user must match transaction parameters.	Client + signed transaction construction

5.2 State machine

Policy lifecycle: DRAFT (client-only) → ACTIVE → PAUSED ↔ ACTIVE → REVOKED. REVOKED is terminal.
 Payment lifecycle: PROPOSED (off-chain) → USER_CONFIRMED (wallet signs) → SETTLED (transaction confirmed) or REJECTED/FAILED (no transfer).

5.3 Execute-payment validation order

1. Load policy, vault, merchant entry, spend tracker, and receipt PDA.
2. Require policy owner as signer for MVP.
3. Reject if policy is not ACTIVE or has expired.
4. Verify provided vault and mint match policy configuration.
5. Verify merchant entry is active and destination token account matches the allowlisted account.
6. Validate amount > 0 and $\leq$ per-transaction limit.
7. Refresh UTC-day spend bucket if the day has changed; reject if daily limit would be exceeded.
8. Reject if receipt/request PDA already exists.
9. Create receipt, increment spending tracker, perform CPI transfer via SPL Token program, emit PaymentExecuted event.

6. Solana Program Specification

Implementation: Anchor (Rust) program deployed to local validator and devnet. The program owns policy state and signs outbound token transfers only through a policy-derived PDA. No private keys are stored by the application.

6.1 Accounts

Account	Seeds / ownership	Key fields	Purpose
Policy	PDA: ["policy", owner, policy_nonce]	owner, usdc_mint, vault, daily_limit, per_tx_limit, expires_at, state, bump	Source of truth for authority and rules
Vault	ATA owned by Policy PDA	mint=USDC; owner=Policy PDA	Holds escrowed USDC
MerchantEntry	PDA: ["merchant", policy, merchant_token_account]	policy, token_account, active, added_at	Policy-local allowlist record
SpendTracker	PDA: ["spend", policy]	day_index, spent_amount	Tracks daily spend
PaymentReceipt	PDA: ["receipt", policy, request_id]	merchant, amount, created_at, tx metadata	Prevents replay and supports history
Owner	System account signer	public key	User who controls policy
USDC Mint / Token Program	Canonical program accounts	mint / program IDs	Enables SPL transfers

6.2 Instructions

Instruction	Signer	Inputs	Effect / key checks
initialize_policy	Owner	nonce, limits, expiry	Creates Policy, Vault, SpendTracker; limits > 0; valid mint; future expiry.
deposit	Owner	amount	Transfers owner USDC to Vault; amount > 0.
add_merchant	Owner	merchant token account	Creates active MerchantEntry; account mint must equal policy USDC mint.
remove_merchant	Owner	merchant token account	Marks entry inactive or closes it; no retroactive effect on receipts.
execute_payment	Owner (MVP)	amount, request_id, reference hash	Checks all policy rules, creates receipt, transfers vault USDC to merchant.
pause_policy	Owner	—	Sets state PAUSED.
resume_policy	Owner	—	Sets state ACTIVE if not expired/revoked.
revoke_policy	Owner	—	Sets terminal REVOKED state.
withdraw	Owner	amount	Transfers unused USDC from Vault to owner ATA; no effect on receipts.

6.3 Events

Event	Fields	Consumer
PolicyCreated	policy, owner, limits, expiry	Frontend and indexer
MerchantUpdated	policy, merchant token account, active	Frontend and audit trail
PaymentExecuted	policy, receipt, request_id, merchant, amount, day_index, reference_hash	Receipt history and merchant webhook
PolicyStateChanged	policy, state, timestamp	Dashboard / alerts
FundsWithdrawn	policy, owner, amount	Audit trail

7. Technical Architecture

The architecture separates intelligence from authority. The AI service can reason and propose. The client constructs a transparent transaction. The Solana program alone decides whether funds leave the vault.

Layer	Responsibilities	Trust boundary
Web application (Next.js / React)	Wallet connection, policy configuration, chat, proposal review, transaction signing, history UI	Untrusted for monetary enforcement; may be modified by a compromised client
Backend / agent service	LLM orchestration, merchant search, quote normalization, proposal creation, order tracking	Must not own user keys; treats external content as untrusted
Quote / merchant adapter	Gets merchant price and payment destination; returns signed or bound order reference	Validate allowlisted merchant identity; prevent destination substitution
Solana Anchor program	Policy checks, PDA vault authority, USDC transfer, receipts, events	Authoritative settlement and rule enforcement
Indexer / database	Caches events, chat metadata, UI history, analytics	Convenience only; chain state is authoritative
Storage	Order descriptions, non-sensitive receipts, optional hashes	Never store seed phrases, private keys, or sensitive payment credentials

7.1 Off-chain API contracts

Endpoint	Request / response	Controls
POST /api/proposals	Request: policyId, userIntent. Response:	Schema validation; rate limit; prompt-

Endpoint	Request / response	Controls
	proposalId, merchant, amount, mint, purpose, requestId, referenceHash, checks[]	injection defenses; server creates no transfer.
GET /api/policies/:id/summary	Returns derived vault balance, daily spend, active merchants, expiry, and event history	Reconcile with RPC; label cached data.
POST /api/merchant/orders	Creates mock order bound to expected merchant, amount and referenceHash	Only mock/allowlisted merchant; idempotency key.
POST /api/webhooks/payment-settled	Receives confirmed transaction/event observation and marks mock order paid	Verify signature/event and confirmation depth; idempotent.

8. AI Agent Design

8.1 Agent responsibilities

- Parse user intent into a purchase request: service, budget, payment urgency, and constraints.
- Discover only approved/mock merchants and retrieve a quote.
- Create a strictly typed proposal; explain why it satisfies the policy.
- Never fabricate a merchant address, transaction confirmation, quote, or policy result.
- Escalate ambiguous requests to the user rather than guessing.

8.2 Tool contract and guardrails

Rule	Implementation
No signing tool	The agent has no wallet private key and no direct capability to submit an unrestricted transfer.
Structured output	Use schema: merchant_id, merchant_token_account, amount_base_units, mint, purpose, quote_expires_at, request_id, reference_hash.
Policy-aware, not policy-authoritative	Agent may call a read-only simulator, but it must describe results as preliminary; program execution is final.
External text isolation	Merchant pages, invoices and user-provided text are data, never system instructions. Prompt injection text is ignored and logged.
Human confirmation	MVP payment proposal must be shown to the owner and signed in wallet.
Auditability	Store model version, prompt template version, tool calls, and proposal hash without storing secrets.

SECURITY PRINCIPLE A model should never be the authorization system. The model can recommend an action; deterministic program checks decide whether money moves.

9. Security, Privacy, and Compliance

9.1 Threat model and mitigations

Threat	Risk	MVP mitigation
Prompt injection from merchant content	Agent is tricked into changing amount/recipient	Typed proposal schema; approved merchant registry; user review; on-chain allowlist.
Malicious or compromised backend	Backend attempts arbitrary transfer	Backend has no user keys; program restricts recipients and limits; owner signature required.
Recipient substitution	Payment goes to attacker token account	MerchantEntry stores canonical token account and mint; program matches exact account.
Replay / duplicate order	Same request pays twice	Receipt PDA derived from unique request_id; duplicate creation fails.
Daily-limit bypass	Multiple small payments exceed cap	SpendTracker updates atomically in same instruction as transfer.
Key compromise	Attacker controls user wallet	Wallet hygiene guidance; pause/revoke; low default limits; no custody.
Program vulnerability	Funds locked or stolen	Unit/integration tests, code review, devnet soak, third-party audit before mainnet.
Indexer mismatch	UI shows stale/incorrect balance	Display confirmation state; refresh from RPC; contract remains authority.

9.2 Security requirements

- Use Anchor account constraints, canonical PDA seeds, checked integer arithmetic, and explicit token/mint/program address validation.
- Use u64 base units for all token amounts. Never use floating point values in the program.
- Require the owner signer on every MVP execute_payment call.
- Validate every account passed to the instruction; do not trust client-provided account relationships.
- Emit events only after successful state changes and transfer CPI.
- Keep program upgrade authority in a multisig before production; publish program ID and IDL.

- Run dependency scanning, linting, unit tests, local-validator integration tests, and a formal security review before mainnet.

9.3 Privacy and compliance

MVP uses devnet and mock merchants only. For production, legal counsel must determine applicable money-transmission, sanctions screening, consumer protection, tax, data protection, and AI-disclosure obligations in target jurisdictions. The product must not represent itself as a bank, investment service, or payment processor without proper licensing and operational controls.

10. Non-functional Requirements

Area	MVP requirement	Measure
Reliability	No false “paid” state before chain confirmation.	100% of displayed settled payments map to a confirmed signature/event.
Performance	Policy read under 3 seconds on normal RPC; transaction UX shows pending state.	p95 frontend policy summary < 3s excluding wallet approval.
Availability	Graceful degradation if LLM or indexer fails.	User can still manage policies and inspect direct chain state.
Accessibility	Keyboard-accessible UI; sufficient contrast; clear transaction/error states.	Manual WCAG-oriented review of critical flows.
Observability	Trace proposal → wallet signature → transaction → receipt.	Correlation IDs and structured logs; no secrets in logs.
Cost	Keep program state compact and limit accounts per payment.	Measure compute units and account rent during devnet tests.

11. Data Model and Retention

Data	Location	Retention / treatment
Policy / merchant / spend / receipt	Solana accounts	Public blockchain data; only store minimum necessary fields and hashes.
Transaction signatures and event cache	Application database	Retain for product history; rebuildable from chain.
Proposal and agent trace	Application database	Retain 90 days for MVP; redact user content; no secrets.
Mock order metadata	Application database	Retain 90 days; referenceHash links order to receipt without exposing full description on-chain.

Data	Location	Retention / treatment
Wallet public key	Client + database if user opts into history	Public identifier; minimize association with unnecessary personal data.

12. Analytics and Success Metrics

Metric	Definition	MVP target
Policy activation rate	Created policies with first deposit ÷ created policies	≥ 60% in usability cohort
Proposal-to-payment conversion	Settled payments ÷ valid proposals	≥ 40% with mock merchant
Policy rejection correctness	Rejected transactions that legitimately violate a rule ÷ total rejected	100% across automated test corpus
Unauthorized-transfer incidents	Transfers that bypass intended policy	0
Payment completion time	Proposal approval to confirmed receipt	p50 < 30 seconds excluding user decision time
User comprehension	Test users who correctly explain limits/revoke behavior	≥ 80% after onboarding

13. Testing and Release Criteria

13.1 Test plan

Layer	Coverage
Program unit tests	Initialize validation, deposits, merchant changes, limits, expiry, daily reset, pause/revoke, withdrawals, replay, bad mint/token program, invalid PDA/account substitution.
Local validator integration	Actual SPL transfers, PDA signing, event emission, concurrent-like transaction ordering, expected failure assertions.
Frontend E2E	Connect wallet, create/fund policy, add merchant, proposal display, wallet rejection, confirmed payment, history, pause/revoke.
Agent evaluation	Ambiguous intent, amount mismatch, unknown merchant, prompt injection, stale quote, user asks to bypass rules.
Security review	Manual authorization/account-constraint review; static analysis; dependency/license review; external audit scope for production.

13.2 Devnet launch gates

- All Must functional requirements pass automated tests.
- 100% test coverage for `execute_payment` success path and each defined rejection condition.
- No critical/high security findings open from internal review.
- Every receipt observed in UI reconciles to an RPC-confirmed event/signature.
- Usability test confirms users understand: funds are in a policy vault, every MVP payment needs signature, and pause/revoke blocks future payments.
- Runbook exists for RPC outage, compromised backend, incorrect UI state, program pause, and emergency user guidance.

13.3 Production gates (out of MVP)

Independent smart-contract audit completed; remediation verified; program upgrade authority secured; monitoring/on-call established; legal/compliance assessment complete; real-merchant integration and dispute/refund policy defined; mainnet USDC mint and token-account handling reviewed; incident response and user communications approved.

14. Delivery Plan

Week	Outcome	Deliverables
1	Foundation	Architecture decisions, UX wireframes, Anchor workspace, local validator, devnet wallet/test-USDC flow.
2-3	Policy contract	Accounts, initialization, deposits, merchant management, basic unit tests.
4	Payment enforcement	Spend tracker, receipt/replay prevention, <code>execute_payment</code> , events, adversarial tests.
5	Frontend policy UX	Wallet integration, policy dashboard/editor, vault funding and merchant controls.
6	Agent proposal flow	Typed tool schema, mock merchant adapter, proposal card, simulator.
7	Settlement UX	Transaction signing, confirmation handling, receipt history, mock fulfillment webhook.
8	Hardening	Error handling, observability, accessibility pass, test expansion, devnet soak.
9	Usability testing	5-8 user sessions, comprehension test, prioritize UX/security fixes.
10	Demo readiness	Seed demo policies, scripted scenarios, architecture walkthrough, release candidate.
11-12	Buffer / next decision	Fixes, documentation, mainnet-readiness decision; do not deploy value without production gates.

15. Open Questions and Decisions Needed

Question	Options	Recommended MVP decision
Payment authorization	Owner signature each time vs. delegated session key	Owner signature each payment. Defer delegation.
Merchant identity	Wallet-only vs. registry + signed quote	Static mock merchant registry, exact token-account allowlist, order reference hash.
Daily window	Rolling 24h vs. UTC calendar day	UTC calendar-day bucket for simple deterministic enforcement.
Policy changes	Allow edit in place vs. create new policy	Allow safe limit/expiry updates only later; MVP creates a policy and manages state/merchants.
Refunds	On-chain refund instruction vs. merchant direct transfer	Merchant direct transfer for mock demo; define refund protocol later.
Agent model	Single hosted model vs. pluggable model providers	Single hosted model behind an adapter; log version and schema validation.

16. Future Roadmap (Post-MVP)

- Delegated session keys with narrower on-chain permissions, expiry, and revocation.
- Recurring payment schedules and subscription caps with explicit renewal windows.
- Merchant registry with verified identities, signed quotes, and escrow/refund flows.
- Multi-user/team policies, approval thresholds, role-based controls, and corporate expense reporting.
- Agent identity and reputation integrations, with stake only after careful economic and legal design.
- Support for service delivery proofs, dispute workflows, and optional private receipt attestations.

Appendix A — Demo Scenario

User Priya creates a “Design Tools” policy: 10 USDC daily limit, 3 USDC per payment, expiry in 30 days. She deposits 20 devnet USDC and allowlists Demo Image API. She asks the agent to buy 2 USDC image credits. The proposal shows the exact merchant token account, amount, purpose, day spend (0 → 2 / 10), and reference hash. Priya approves in Phantom. The program verifies all rules, transfers 2 USDC from the vault, creates the receipt PDA, and emits PaymentExecuted. The dashboard marks the order paid only after confirmation. Priya pauses the policy; subsequent execution attempts fail on-chain.

Appendix B — Definition of Done

- A user can create, fund, use, pause, revoke, and withdraw from a policy on devnet.
- The only available payment recipient is an explicitly allowlisted mock merchant token account.
- Every payment shows a confirmed signature and receipt record.
- Automated tests prove each core safety rule cannot be bypassed through direct program calls.
- The AI agent produces structured proposals and never has access to an unrestricted signing capability.
- The repository contains setup instructions, threat model, test commands, IDL, and demo script.

Appendix C — Glossary

Term	Meaning
Anchor	A Rust framework that simplifies Solana program development and account validation.
PDA	Program Derived Address: a deterministic account address controlled by a program, not a private key.
SPL token	Solana Program Library token standard; USDC is represented as an SPL token.
Vault	The policy-owned token account that holds funds subject to program rules.
Request ID	A unique proposal/order identifier used to derive a receipt and prevent replay.